

Arquitectura y Especificaciones de la Plataforma IIP

Juan José Martínez Guerrero¹

1: IIP Platform | Lead Maintainer

<https://github.com/SpanishSyntax>

Abstract

La plataforma del Índice de Innovación Pública (IIP) constituye un ecosistema de gestión de datos de alto rendimiento, diseñado como una arquitectura de microservicios orientada al dominio (DDD) para servir como el motor central de inteligencia y trazabilidad de la Veeduría Distrital. El sistema implementa una arquitectura basada en contenedores Docker que integra componentes especializados: autenticación, lógica central (Core), persistencia (Alembic/Seeders), almacenamiento persistente en PostgreSQL y un proxy inverso Nginx. Desarrollada bajo el patrón Fast API con capas de servicios, unidades de trabajo (UOW) y repositorios, la solución centraliza el ciclo de vida completo de las versiones 2019, 2021, 2023 y 2025 del IIP, abarcando desde la gestión de cuestionarios y respuestas hasta el procesamiento analítico de resultados. Esta infraestructura está diseñada como una solución escalable y abierta, preparada para la integración futura con motores de vectorización, sistemas de Generación Aumentada por Recuperación (RAG) y protocolos de Model Context Protocol (MCP), facilitando tanto la captura de nuevas propuestas y la gestión de procesos de evaluación, como la democratización de datos a través de portales de datos abiertos.

1 Introducción

La plataforma del índice de Innovación Pública (desde ahora IIP) se erige como un sofisticado ecosistema tecnológico, concebido como un hub de datos multidominio de alta disponibilidad, diseñado específicamente para satisfacer las necesidades analíticas y de gobernanza de la Veeduría Distrital. Este sistema no solo actúa como un repositorio centralizado, sino como un motor de procesamiento inteligente capaz de orquestar la complejidad inherente a los datos distritales, integrando de manera fluida la gestión dinámica de formularios, la administración de actores estratégicos y sistemas de evaluación robustos.

Bajo una arquitectura de microservicios estrictamente desacoplada, la plataforma garantiza una separación de responsabilidades que optimiza el ciclo de vida del software, permitiendo que los servicios de autenticación, la lógica de negocio central y la capa de persistencia operen como entidades independientes, aunque perfectamente cohesionadas. Todo este desarrollo está consolidado bajo una estrategia de monorepo, la cual permite la gestión unificada del código fuente, facilitando el intercambio de lógica a través de una biblioteca interna compartida. Esta infraestructura técnica, robusta y escalable, ha sido diseñada con una visión a largo plazo, garantizando que el sistema sea capaz de evolucionar desde una solución de gestión administrativa hacia un núcleo tecnológico preparado para la integración de sistemas de vectorización, arquitecturas RAG, y protocolos de comunicación de última generación como MCP, consoli-

dándose así como la infraestructura de datos definitiva para la innovación pública.

1.1 Propósito

El propósito fundamental de la plataforma IIP es democratizar el acceso y la gestión del conocimiento derivado del Índice de Innovación Pública (IIP), funcionando como la columna vertebral de datos para la Veeduría Distrital. El alcance del sistema abarca la consolidación histórica y analítica de todas las versiones del índice (2019, 2021, 2023 y 2025), transformando una estructura de datos fragmentada en un modelo de información coherente, versionable y auditable.

En términos operativos, la plataforma está diseñada para satisfacer tres pilares críticos:

1. Gestión Integral del Ciclo de Vida: Desde la captura de nuevas respuestas y la gestión de actores, hasta la evaluación técnica realizada por el colegio calificador.
2. Interoperabilidad de Datos: Actuar como fuente única de verdad para la publicación de datos abiertos, facilitando el consumo analítico externo y asegurando la transparencia gubernamental.
3. Extensibilidad Inteligente: Servir como base de datos para sistemas avanzados, incluyendo la futura integración de servicios de RAG (Generación Aumentada por Recuperación) y agentes de IA, permitiendo consultas semánticas complejas sobre todo el histórico de resultados del IIP.

1.2 Alcance

1.3 Audiencia destinada

La plataforma IIP está diseñada para servir a un ecosistema diverso de usuarios, cada uno con necesidades específicas de interacción y niveles de acceso diferenciados:

- **Entidades Distritales (Sujetos de Evaluación):** Son los responsables de la carga de información. A través de funcionarios autorizados, representan a sus entidades para responder al IIP. El sistema garantiza que esta escritura sea trazable, auditable y centralizada, convirtiéndose en su herramienta oficial de reporte ante la Veeduría.
- **Niveles de Decisión Estratégica (Secretaría General y Cabezas de Sector):** Estos actores utilizan los resultados del IIP como el principal insumo de diagnóstico. Su acceso está enfocado en la analítica de alto nivel para identificar brechas, priorizar inversiones y dirigir recursos hacia áreas que requieren mejoras sustanciales en sus capacidades de innovación pública.
- **Ecosistema de Innovación (Labs Distritales como iBo):** Estos laboratorios actúan como catalizadores. Utilizan la data cruda y los resultados procesados para diseñar intervenciones, experimentos y estrategias de acompañamiento técnico a las entidades, transformando los puntajes bajos en planes de mejora concretos.
- **Organismos de Control:** Utilizan el sistema como la fuente de verdad técnica para la generación de recomendaciones, auditorías y el seguimiento a políticas públicas transversales, incluyendo el cumplimiento del CONPES 04 y otros marcos normativos vigentes.
- **Comunidad Académica, Observatorios y ONGs:** Actores que buscan democratizar el dato. Acceden a los resultados mediante portales de visualización, herramientas de exportación y consultas semánticas mediante IA, fomentando el escrutinio público y la investigación longitudinal.
- **Arquitectos y Desarrolladores:** Personal técnico encargado de la integración del IIP con sistemas externos, servicios de RAG o protocolos de comunicación MCP, garantizando que el sistema evolucione como una pieza fundamental de la infraestructura digital del Distrito.

1.4 Visión general de la plataforma

La plataforma IIP se posiciona como el ecosistema tecnológico de referencia para la gestión, procesamiento y democratización de los datos asociados al Índice de Innovación Pública de Bogotá. Más allá de actuar como un repositorio estático, la plataforma se ha concebido bajo tres pilares fundamentales que transforman la forma en que el Distrito gestiona su innovación:

1. **Centralización de la Fuente de Verdad:** Históricamente, la información del IIP se encontraba dispersa en estructuras no normalizadas. La plataforma IIP consolida todas las versiones históricas (2019-2025) en un modelo

de datos único, auditable y versionable, garantizando que tanto los organismos de control como los tomadores de decisiones operen sobre la misma base de datos íntegra.

2. **Motor de Inteligencia y Trazabilidad:** La arquitectura implementa un flujo transaccional donde cada interacción —desde la respuesta inicial de una entidad hasta la calificación final por parte del colegio calificador— es registrada y validada. Este nivel de trazabilidad permite no solo la transparencia, sino también la creación de analíticas complejas que identifican patrones de mejora en tiempo real.
3. **Ecosistema Abierto y Escalable:** La plataforma está diseñada para trascender su función administrativa. Al exponer la data cruda a través de APIs documentadas, permite que laboratorios como iBo, investigadores académicos y herramientas de IA (RAG/MCP) se integren de manera nativa. Esto asegura que el sistema no sea una «caja negra», sino un motor abierto que se adapta tanto a los reportes de política pública (como el CONPES 04) como a las necesidades de análisis de datos a futuro.

En esencia, la plataforma IIP es la infraestructura habilitante que convierte la complejidad administrativa en evidencia técnica, facilitando que las entidades, los laboratorios de innovación y los entes de control no solo midan, sino que transformen el desempeño del sector público en Bogotá.

1.5 Principios de diseño

La arquitectura de la plataforma IIP ha sido fundamentada sobre cinco principios rectores que garantizan la integridad, escalabilidad y transparencia del ecosistema:

1.5.1 Desacoplamiento de Dominios (Arquitectura Hexagonal/DDD):

La lógica de negocio reside en una capa central pura, independiente de frameworks web o bases de datos específicas. Esto permite que el sistema evolucione (cambiar una base de datos o migrar a una nueva versión de Python) sin alterar las reglas de validación del Índice de Innovación Pública.

1.5.2 Consistencia como Fuente de Verdad:

El sistema opera bajo el principio de centralización de la persistencia. Al utilizar SQLAlchemy y Alembic, garantizamos que cualquier dato, desde un histórico de 2019 hasta una nueva respuesta, mantenga la integridad relacional. No existen datos duplicados ni versiones divergentes de la realidad.

1.5.3 Reproducibilidad Total (Infraestructura como Código):

El despliegue no debe depender de configuraciones manuales («serendipia de servidor»). Mediante el script `launcher.sh`, el sistema garantiza que cualquier entorno (ya sea desarrollo, pruebas o producción) pueda levantarse

desde cero con una configuración consistente, incluyendo esquemas y semillas de datos.

1.5.4 Seguridad por Diseño:

La confianza es crítica para una herramienta de la Veeduría. La seguridad no se añade al final; se implementa mediante capas: autenticación JWT con firma ED25519, hashing de contraseñas con Argon2, y gestión de secretos aislada para evitar la exposición de credenciales en el repositorio.

1.5.5 Apertura para la Extensibilidad (Data-First):

El diseño anticipa el futuro. La plataforma no está cerrada; se construye con el principio de «API-first», permitiendo que sistemas externos —ya sean herramientas de visualización, agentes de IA para análisis semántico (RAG) o protocolos de integración (MCP)— consuman los datos de forma estructurada y controlada sin comprometer la seguridad.

2 Arquitectura del sistema

La plataforma IIP adopta un diseño modular basado en microservicios, eliminando la complejidad de los monolitos tradicionales mediante una estrategia de monorepo gestionada por workspaces. Esta arquitectura está diseñada para maximizar la independencia de los dominios funcionales (Autenticación, Lógica de Negocio y Persistencia), asegurando al mismo tiempo una gobernanza centralizada del código. La infraestructura se apoya en la contenerización para garantizar la portabilidad y la escalabilidad granular, permitiendo que cada componente evolucione de manera autónoma sin comprometer la integridad del ecosistema.

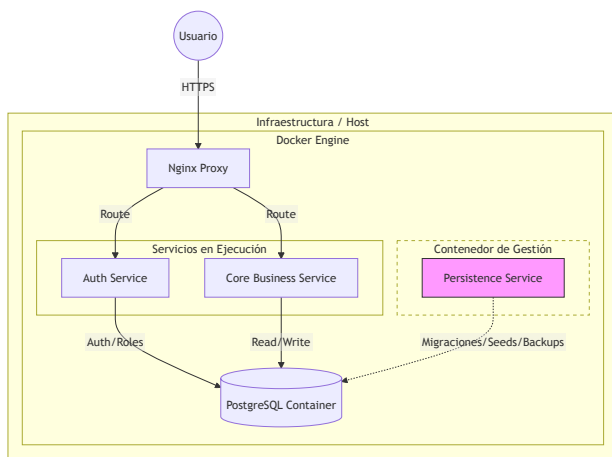


Figura 1: Arquitectura de despliegue.

2.1 Arquitectura de alto nivel

A nivel técnico, el sistema se organiza bajo el principio de separación de responsabilidades, utilizando una arquitectura de capas basada en Domain-Driven Design (DDD). Esta estructura se divide en dos planos fundamentales: el despliegue de infraestructura y la organización lógica del código.

En el plano operativo, un proxy inverso Nginx actúa como la única puerta de entrada hacia la plataforma. Este componente gestiona el enrutamiento seguro de las peticiones del usuario hacia los servicios correspondientes (Auth para identidad y Core para negocio), los cuales interactúan con un motor centralizado de PostgreSQL. Por su parte, el servicio de Persistence opera como un componente de gestión fuera del flujo crítico de usuario, dedicado exclusivamente a la integridad del ciclo de vida de los datos (migraciones, poblamiento y respaldos), tal como se detalla en Figura 1.

En el plano de desarrollo, el uso de un monorepo nos permite que servicios independientes —como Auth, Core y Persistence— compartan una capa de utilidades y modelos definida en el módulo Shared. Como se ilustra en Figura 3, esta dependencia garantiza una consistencia semántica en todo el sistema y evita la duplicidad de lógica, permitiendo que las entidades de dominio sean coherentes en toda la plataforma.

El sistema se despliega mediante una arquitectura basada en contenedores Docker, orquestada para garantizar el aislamiento y la escalabilidad de cada componente. Un proxy inverso Nginx actúa como puerta de enlace, gestionando el enrutamiento del tráfico hacia los servicios correspondientes y asegurando una comunicación segura. La lógica interna sigue el patrón de diseño Domain-Driven Design (DDD), implementando una arquitectura de capas que organiza el código en servicios, unidades de trabajo (UOW) y repositorios, asegurando que la lógica de negocio permanezca desacoplada de los detalles técnicos de persistencia. Esta disposición permite una evolución independiente de cada módulo, desde la capa de autenticación hasta el motor de procesamiento de datos gestionado por el servicio de core y el motor de base de datos PostgreSQL.

2.2 Decisiones arquitectónicas

2.3 Visión general de los servicios

2.4 Estructura de monorepo

2.5 Stack tecnológico

El sistema está construido sobre un stack moderno orientado al rendimiento y la seguridad transaccional:

1. **Runtime:** Python 3.12.13
2. **Framework:** FastAPI
3. **ORM & DB:** SQLAlchemy 2.0 con PostgreSQL
4. **Migraciones:** Alembic
5. **gestión de paquetes:** uv with workspace support
6. **Seguridad:** Argon2 para hashing y ED25519 para JWT

Para un desglose detallado de las dependencias en materia de librerías del proyecto, por favor remítase a los `pyproject.toml` correspondientes a cada contenedor y al `pyproject.toml` global del proyecto.

3 Arquitectura de despliegue

- 3.1 Visión general de la infraestructura
- 3.2 Arquitectura Docker
- 3.3 Proxy inverso (Nginx)
- 3.4 Comunicación entre servicios
- 3.5 Gestión de configuración
- 3.6 Variables de entorno
- 3.7 Gestión de secretos

4 Arquitectura de dominio

La plataforma IIP adopta un diseño modular basado en microservicios, eliminando la complejidad de los monolitos tradicionales mediante una estrategia de monorepo gestionada por **workspaces**. Esta arquitectura está diseñada para maximizar la independencia de los dominios funcionales (Autenticación, Lógica de Negocio y Persistencia), asegurando al mismo tiempo una gobernanza centralizada del código. La infraestructura se apoya en la contenerización para garantizar la portabilidad y la escalabilidad granular, permitiendo que cada componente evolucione de manera autónoma sin comprometer la integridad del ecosistema.

La arquitectura se organiza bajo el principio de separación de responsabilidades, utilizando un diseño de capas basa-

Servicio	Responsabilidad	Entidades Clave
Nginx	Proxy inverso y enrutamiento	Configuración, Certificados
Auth	Gestión de identidad y seguridad	User, Role, RefreshSession
Core	Lógica de negocio y procesamiento de dominio	Form, Submission, Actor, Grade
Persistence	Ciclo de vida de BD, migraciones y carga	Alembic, Seeder, Populator
Shared	Lógica común, modelos ORM y utilidades	Base, AccessContext, Hashing

Tabla 1: Componentes del sistema y sus responsabilidades.

do en **Domain-Driven Design** (DDD). Esta estructura se divide en dos planos fundamentales: el despliegue de infraestructura y la organización lógica del código. En el plano operativo, el sistema garantiza que la lógica de negocio permanezca aislada de los detalles técnicos, mientras que el plano de desarrollo aprovecha el monorepo para compartir tipado y utilidades a través de una librería centralizada, garantizando una consistencia semántica absoluta en todo el sistema.

4.1 Diseño guiado por el dominio (DDD)

El servicio Core implementa una arquitectura basada en Domain-Driven Design [1] (DDD), organizada en capas que separan las responsabilidades de presentación, aplicación, dominio e infraestructura. Esta organización permite mantener la lógica de negocio desacoplada de los mecanismos de persistencia y comunicación, facilitando la mantenibilidad, la extensibilidad y las pruebas unitarias.

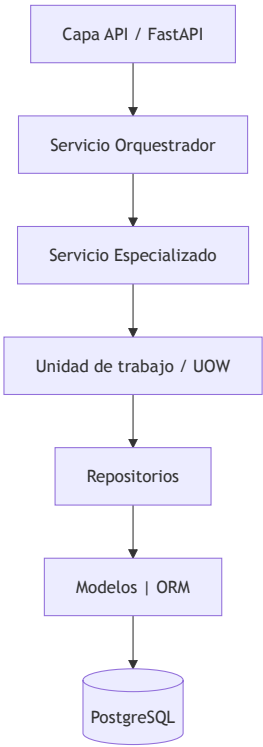


Figura 2: Arquitectura de capas orientada al dominio (DDD).

1. **Capa API:** Expone los endpoints REST mediante FastAPI y actúa como punto de entrada al sistema. Su responsabilidad se limita a la validación de solicitudes, autenticación y serialización de respuestas.
2. **Servicio Orquestrador:** Coordina el flujo de ejecución de los casos de uso. Esta capa encapsula la lógica de alto nivel, delegando las operaciones específicas en servicios especializados sin contener reglas de negocio propias.
3. **Servicios Especializados:** Implementan la lógica de negocio asociada a cada dominio funcional, como la gestión de formularios, respuestas o auditorías. Estos

servicios utilizan la Unidad de Trabajo para ejecutar operaciones transaccionales de forma consistente.

4. **Unidad de Trabajo y Repositorios:** La Unidad de Trabajo (UOW) administra el ciclo de vida de las transacciones y garantiza la atomicidad de las operaciones. Los repositorios abstraen el acceso a los datos, desacoplando la lógica de negocio de la tecnología de persistencia utilizada.
5. **Modelos ORM y Base de Datos:** La capa de persistencia emplea modelos definidos con SQLAlchemy ORM para mapear las entidades del dominio a la base de datos PostgreSQL, proporcionando una interfaz orientada a objetos sobre el almacenamiento relacional.

Esta separación de responsabilidades facilita la evolución independiente de cada capa, mejora la mantenibilidad del sistema y simplifica la implementación de pruebas unitarias al reducir el acoplamiento entre la lógica de negocio y la infraestructura.

4.2 Arquitectura en capas

El sistema se organiza en un flujo unidireccional de dependencias donde las capas internas (Dominio) son independientes de las capas externas (Infraestructura).

- **Dominio:** Contiene las entidades, los objetos de valor y las reglas de negocio puras. Es la capa más protegida.
- **Aplicación:** Orquesta las reglas de negocio para ejecutar casos de uso específicos.
- **Infraestructura:** Implementa los detalles técnicos (bases de datos, frameworks web, clientes externos).

4.3 Servicios de aplicación

Los servicios de aplicación actúan como coordinadores de alto nivel. Su responsabilidad es capturar la solicitud desde la capa API, validar el contexto de seguridad inyectado en el JWT y orquestar la llamada a la capa de dominio o a los servicios especializados. No ejecutan lógica de negocio per se, sino que aseguran que el flujo de datos sea consistente entre las entradas del usuario y las entidades del sistema.

4.4 Repositorios

Los repositorios abstraen el acceso a la base de datos, tratando la persistencia como si fuera una colección de objetos en memoria. Cada agregado del dominio cuenta con su propio repositorio, lo que permite intercambiar la implementación técnica (SQLAlchemy, almacenamiento en caché, etc.) sin afectar los servicios de aplicación. El contrato se define mediante interfaces en el dominio, cumpliendo con el **Principio de Inversión de Dependencias**.

4.5 Unidad de trabajo (Unit of Work)

La Unidad de Trabajo es el mecanismo que garantiza la consistencia transaccional. Implementa el patrón **Commit/Rollback** de forma centralizada:

1. **Inicio:** Crea una transacción explícita al comenzar el caso de uso.
2. **Ejecución:** Coordina múltiples repositorios durante la operación.
3. **Finalización:** Aplica `commit` solo si todas las operaciones fueron exitosas, o `rollback` en caso de cualquier excepción, asegurando que el sistema nunca quede en un estado intermedio inconsistente.

4.6 Librería compartida

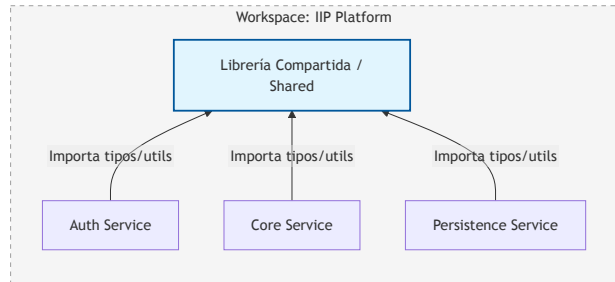


Figura 3: Dependencias de módulos dentro del monorepo.

4.7 Ciclo de vida de la solicitud

El ciclo de vida de una petición sigue un flujo estricto para garantizar la seguridad:

1. **Ingreso:** La solicitud llega al Proxy Nginx, donde se verifica el formato del JWT.
2. **Autenticación:** El middleware de FastAPI en el servicio `Core` decodifica y verifica la firma (ED25519) y la integridad del `AccessContext`.
3. **Autorización:** Se inyecta la dependencia `AccessContext` que valida si el `sub` y los `assignments` tienen permisos sobre el `entity_id` solicitado.
4. **Orquestación:** El Servicio de Aplicación recibe el comando y abre la **Unidad de Trabajo**.
5. **Persistencia:** La UOW delega en los **Repositorios** para recuperar o guardar entidades.
6. **Respuesta:** La UOW confirma la transacción y la capa API serializa el resultado hacia el cliente.

5 Arquitectura de datos

5.1 Visión general de la base de datos

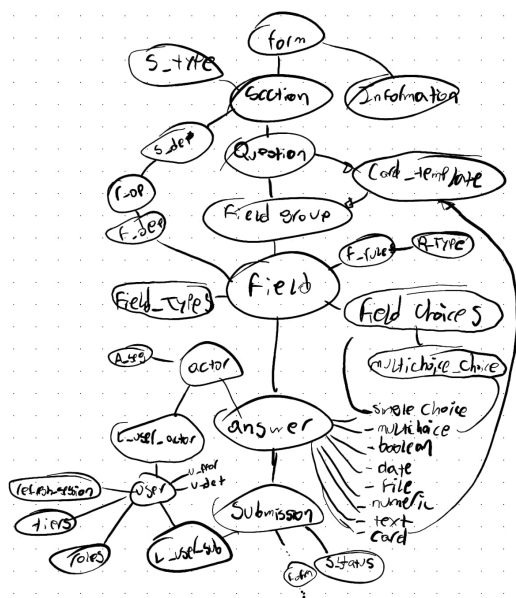


Figura 4: Arquitectura de flujo y dependencias del sistema de persistencia.

Packages/shared/src/shared/models

- actors.py
- audit.py
- auth.py
- files.py
- forms.py
- grading.py
- __init__.py
- interactions.py
- links.py
- reference.py
- rules.py
- submissions.py
- targets.py

5.2 Organización del esquema

5.3 Entidades de dominio

5.4 Relaciones

5.5 Versiones históricas de IIP

5.6 Integridad de datos

5.7 Auditoría

6 Persistencia

La capa de persistencia constituye la base de la integridad operacional de la plataforma IIP. Su diseño no se limita al almacenamiento relacional, sino que implementa un ecosistema de gestión automatizada que garantiza la trazabilidad, consistencia y recuperabilidad de la información a lo largo de todo el ciclo de vida del dato. A través de una serie de componentes especializados —orquestados por el servicio persistir—, el sistema gestiona desde la creación dinámica de esquemas y la evolución del modelo mediante migraciones versionadas, hasta la ingesta inteligente de volúmenes históricos y la salvaguarda de la base de datos mediante respaldos automáticos.

Este diseño modular asegura que el estado del sistema sea siempre auditable y replicable. Al desacoplar las tareas de mantenimiento de la base de datos de la lógica de negocio activa, el sistema permite operaciones de administración (como migraciones de esquema o restauración de respaldos) de forma aislada y controlada, cumpliendo con los requisitos de robustez y disponibilidad exigidos por la Veeduría Distrital.

6.1 Creación de DB schemas

Este módulo automatiza la creación de esquemas de base de datos en PostgreSQL durante el despliegue inicial del sistema, inspeccionando dinámicamente las clases del modelo de datos para garantizar la existencia de las estructuras necesarias. El script de persistencia extrae de manera única los nombres de los esquemas definidos en la clase `TargetTable` y sus clases base mediante introspección de código, identificando cada atributo que implemente el tipo `TableInfo`.

Posteriormente, abre una sesión síncrona con la base de datos y ejecuta de forma iterativa comandos seguros de creación de esquemas que previenen errores por duplicación. El flujo maneja de forma independiente cada confirmación o reversión ante fallos y genera un informe final en el registro del sistema detallando las estructuras creadas de manera exitosa y las fallidas.

El script `launcher.sh` ejecuta automáticamente este proceso de inicialización al detectar la bandera `--setup`, simplificando la configuración del entorno cuando el sistema se despliega por primera vez en un servidor virgen.

i Nota: Nota: La ejecución exitosa mediante la bandera `--setup` requiere que las credenciales y variables de entorno de la base de datos estén correctamente configuradas, y que el motor de PostgreSQL esté activo y aceptando conexiones.

6.2 Alembic

La configuración de Alembic permite gestionar el ciclo de vida de la base de datos, incluyendo la carga dinámica de modelos y la definición de una tabla personalizada para las migraciones, `alembic_automatic_version`. Para asegurar la persistencia y sincronización entre el host y el contenedor `app_persist`, el volumen de versiones de Alembic debe montarse en `docker-compose`, reflejando la estructura necesaria de archivos en `Persistence/src/migrator/alembic/versions`.

```
persist:
...
volumes:
  # Huesped : Contenedor
  - ".Persistence/src/migrator/alembic/versions:/api/migrator/alembic/versions"
  # Cambie sólo la ruta en huésped, NO cambie ruta en contenedor.
```

El montaje del volumen de Alembic es obligatorio, y si el directorio `versions` en el host está vacío o no coincide con el estado actual de los modelos al levantar el servicio `persist`, las migraciones automáticas fallarán al no poder contrastar la metadata con el historial de revisiones previo.

El sistema facilita la creación de nuevas migraciones mediante el script `launcher.sh`. Este mecanismo automatiza la detección de cambios entre la definición actual de los modelos (`Packages/shared/models`) y el estado real de la base de datos, generando los archivos de migración necesarios de forma estructurada.

Para crear una nueva revisión tras modificar los modelos, utilice la bandera `-revision` especificando una descripción breve de los cambios:

```
./launcher.sh --revision "add_user_bio_field"
```

El script ejecuta internamente el comando de alembic `revision --autogenerate` dentro del contenedor `persist`. Esto asegura que el código generado sea coherente con el entorno de ejecución, capturando automáticamente cualquier adición, modificación o eliminación de columnas o tablas.

i Nota: Al utilizar `-revision`, el sistema crea un nuevo archivo de migración en `Persistence/src/migrator/alembic/versions/`. Es recomendable revisar el contenido de este archivo antes de aplicar la migración en

entornos de producción para asegurar que Alembic ha detectado correctamente los cambios deseados.

6.3 Poblado de sistema (seeds)

Este módulo automatiza la carga de datos maestros e iniciales (seeds) en la base de datos tras asegurar la existencia de los esquemas y aplicar las migraciones correspondientes. El mecanismo implementado en `seeder.py` escanea dinámicamente el directorio interno `seeds/`, ordenando alfabéticamente los archivos encontrados para garantizar una secuencia de ejecución predecible y respetar las dependencias relacionales subyacentes. Utilizando el módulo `importlib.util` de Python, el script realiza una carga reflexiva de cada archivo `.py`, busca de forma explícita una función ejecutable llamada `upgrade()` y la invoca de manera aislada dentro de un bloque controlado de excepciones. El flujo captura errores individuales por archivo para evitar que un fallo en un set de datos interrumpa todo el proceso de inicialización, generando un registro detallado en el `logger` y volcando la traza completa (`traceback`) en la consola ante cualquier eventualidad.

Al igual que los módulos previos de persistencia, este componente es invocado automáticamente por el script `launcher.sh` al ejecutar la bandera `--setup` durante el despliegue en un entorno virgen, asegurando que las tablas base queden completamente pobladas.

Para asegurar el correcto orden de inserción (por ejemplo, registrar roles antes que usuarios), el directorio debe mantener una nomenclatura secuencial estricta tal como se ilustra en la siguiente estructura física:

```
Persistence/src/seeder
├── seeder.py
├── seeds
│   ├── 00a_seed_log_action_types.py
│   ├── 00b_seed_file_types.py
│   ├── 00c_seed_roles.py
│   ├── 00d_seed_user_tiers.py
│   ├── 10a_seed_users.py
│   ├── 1b_seed_rule_types.py
│   ├── 1c_seed_relational_operators.py
│   ├── 1f_seed_submission_status_types.py
│   ├── 30a_seed_field_types.py
│   ├── 33a_seed_notification_types.py
│   ├── 33b_seed_comment_types.py
│   └── __init__.py
```

i Nota: Nota: Cualquier script de inicialización nuevo que se añada a la carpeta `seeds/` debe implementar obligatoriamente la función `upgrade()`. Se recomienda seguir el patrón numérico/alfabético prefijado (`00a_`, `10a_`) para controlar de forma explícita el orden de carga y evitar fallos por restricciones de llave foránea en la base de datos.

6.4 Poblado de históricos (populator)

Este componente gestiona la ingesta masiva de datos históricos y estructuras complejas en el sistema a través de la API pública de los microservicios, en lugar de realizar inserciones directas en el motor de persistencia. El módulo se orquesta de manera asíncrona mediante `asyncio` y `httpx`, conectándose con el servicio de autenticación (`api_auth`) y el núcleo del sistema (`api_core`) utilizando variables de entorno para resolver los endpoints internos de la red de Docker. El flujo extrae las credenciales iniciales de un archivo TOML administrado mediante secretos de Docker (`/run/secrets/users_file`), priorizando cuentas de nivel `root` para autenticarse, obtener el token Bearer correspondiente e inyectarlo automáticamente en las cabeceras de las peticiones. Para mitigar fallas en tareas de larga duración, la capa del cliente (`ServiceClient`) implementa mecanismos de re-autenticación automática que refrescan el token de acceso si este expira a mitad de una transacción.

La extracción de la data histórica se delega a un conector dedicado (`GitHubConnector`) que descarga los conjuntos de datos crudos o estructuras estructuradas directamente desde repositorios remotos utilizando un token de acceso seguro provisto como secreto de Docker (`/run/secrets/github_token_seeds`). Cada registro recuperado pasa por una capa estricta de validación local mediante esquemas de Pydantic antes de despacharse hacia la API pública de `api_core`. La transferencia de datos se realiza tanto en registros individuales como en procesamiento por lotes que comparten el mismo grupo de conexiones para optimizar el rendimiento. Al procesar las solicitudes mediante peticiones HTTP estándar (`POST`), el sistema garantiza que toda la lógica de negocio, validaciones relacionales y disparadores de eventos del backend se apliquen correctamente a la data histórica, tal como ocurriría con la interacción regular de un usuario.

La estructura física del módulo organiza las tareas en el directorio `pops/` de forma secuencial, incluyendo scripts específicos de migración y plantillas locales en formatos estructurados (CSV y XLSX) dentro de subdirectorios de trabajo:

```
Persistence/src/populator
├── pops
│   ├── 10a_seed_sectors.py
│   ├── 10b_seed_entities.py
│   ├── 11a_seed_section_types.py
│   ├── 11b_seed_forms.py
│   ├── 11c_seed_sections.py
│   ├── 11d_seed_questions.py
│   ├── 11e_seed_loop_questions.py
│   ├── 11f_seed_card_templates.py
│   ├── 11g_seed_field_groups.py
│   ├── 11h_seed_fields.py
│   ├── 11i_seed_field_choices.py
│   └── 12a_seed_field_dependencies.py
```

```
├── 12b_seed_field_rules.py
├── 13a_seed_grading_criteria.py
├── __init__.py
├── jhonatan
│   ├── actors.actor_segments_template.csv
│   ├── actors.actors_template.csv
│   ├── Entidades.csv
│   └── Estructura_IIP.xlsx
└── populator.py
```

i Nota: Nota: A diferencia de los módulos de esquemas, migraciones y datos maestros, el proceso de población masiva histórica no se ejecuta de forma mandatoria con la bandera `--setup` en sistemas limpios si los secretos o conectores externos de GitHub no están mapeados. Este componente requiere que tanto el contenedor de autenticación como el servicio núcleo estén completamente activos, saludables y con la base de datos ya estructurada.

6.5 Backup y Restore

Este sistema gestiona los respaldos de la base de datos PostgreSQL mediante un volumen en Docker Compose que vincula el directorio local `./Persistence/src/backups` con la ruta interna `/backups` del servicio `persister`.

```
persister:
...
volumes:
- "./Persistence/src/backups:/backups"
```

La administración se automatiza a través del script `launcher.sh` empleando dos banderas principales: `--backup`, que ejecuta de forma no interactiva un `pg_dump` nombrando el archivo con una marca de tiempo para evitar sobrescrituras, y `--restore`, que requiere el nombre del archivo `.sql` como argumento e inyecta la variable `PGPASSWORD` internamente para autenticar de manera automática la restauración mediante `psql`.

Dado que el proceso de restauración sobrescribe los datos actuales de la base de datos, se recomienda ejecutar un respaldo preventivo antes de realizar cualquier importación.

i Nota: Nota: Se debe asegurar que el directorio local `./Persistence/src/backups` cuente con los permisos de lectura y escritura correctos para que el contenedor pueda almacenar los archivos.

La gestión de respaldos se centraliza en el script de automatización utilizando las siguientes banderas:

6.5.1 Crear un Respaldo (`--backup`)

Genera una copia de seguridad en caliente de la base de datos PostgreSQL utilizando la herramienta `pg_dump`.

El comando se ejecuta de forma no interactiva (-T) y almacena el archivo resultante usando una marca de tiempo (`TIMESTAMP`) para evitar la sobrescritura de respaldos anteriores.

```
--backup)
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
BACKUP_DIR="./Persistence/src/backups"

mkdir -p "$BACKUP_DIR"

echo "Creating database backup inside
$BACKUP_DIR..."

if docker compose exec -T db sh -c "pg_dump -
U \"${USER_VAR}\" -d \"${DB_VAR}\" > /backups/
db_backup_${TIMESTAMP}.sql"; then
    echo "✅ Backup completed successfully:
$BACKUP_DIR/db_backup_${TIMESTAMP}.sql"
else
    echo "❌ Backup failed."
fi
;;
```

6.5.2 Restaurar un Respaldo (`--restore`)

Permite restaurar un archivo .sql específico cargándolo directamente en el motor de la base de datos mediante psql. El script requiere el nombre del archivo como parámetro e inyecta la variable `PGPASSWORD` de forma interna para realizar la autenticación automática sin solicitar credenciales en la terminal.

```
--restore)
INPUT_FILE="${1:-}"
if [ -z "$INPUT_FILE" ]; then
    echo "Usage: ./launcher.sh --restore
<backup_filename.sql or path>"
    echo "Example: ./launcher.sh --restore
db_backup_20260618_232418.sql"
    exit 1
fi

FILE_NAME=$(basename "$INPUT_FILE")

echo "Restoring database from $FILE_NAME..."

# Inject PGPASSWORD so Postgres authenticates
# automatically without prompting
if docker compose exec -T db sh -c
"PGPASSWORD=\"\${POSTGRES_PASSWORD}\" psql -
h db -U \"${USER_VAR}\" -d \"${DB_VAR}\" < /
backups/$FILE_NAME"; then
    echo "✅ Restore completed successfully."
else
    echo "❌ Restore failed. Make sure $FILE_NAME
exists in your backups directory."
fi
;;
```

7 Autenticación y Autorización

El servicio de autenticación actúa como el pilar de seguridad de la plataforma. Su función es establecer una identidad verificable y un contexto de seguridad **stateless** que acompaña cada solicitud hacia el servicio Core, garantizando escalabilidad y resistencia frente a accesos no autorizados.

```
claims={
  "sub": str(user_id),
  "username": str(username),
  "token_type": str(token_type),
  "iat": int(now.timestamp()),
  "exp": int(exp.timestamp()),
}

jti: UUID | None = None
if token_type == "refresh":
    jti = UUID(str(uuid7()))
claims["jti"] = str(jti)
```

7.1 Arquitectura de Identidad y Acceso

El sistema gestiona la seguridad a través de cuatro dimensiones que operan de forma desacoplada para asegurar un control granular:

1. **Identidad (Auth Service):** Gestión centralizada de credenciales mediante `Argon2id` y emisión de tokens.
2. **Capacidad Operativa (UserTier):** Define los límites de recursos (cuotas de almacenamiento, límites de API) que el sistema aplica al usuario.
3. **Gobernanza Funcional (RBAC):** Definición de roles (ej. `Editor`, `Grader`) que determinan las capacidades técnicas del usuario.
4. **Alcance (ReBAC):** Vinculaciones dinámicas en tablas de enlace (`UserActorLink`, `UserSubmissionLink`) que definen sobre qué objetos específicos se aplican los roles.

7.2 Autenticación basada en JWT

La plataforma utiliza JSON Web Tokens (JWT) firmados con **ED25519**. La firma criptográfica garantiza la inalterabilidad; cualquier modificación invalida el token, siendo verificado por cada microservicio mediante la clave pública.

7.2.1 Estructura del Contexto (Claims)

El token inyecta un objeto de contexto que define las capacidades operativas y funcionales del usuario, evitando consultas constantes a la base de datos:

```
claims={
  "sub": str(user.id),
  "context": {
    "tier": "PREMIUM",
    "assignments": [
      {"entity_id": "uuid-1", "role":
"editor"}],
```

```

    {"entity_id": "uuid-2", "role": "grader"}
  },
  "iat": int(now.timestamp()),
  "exp": int(exp.timestamp()),
}

```

7.3 Autorización: Del RBAC al ReBAC

La autorización no es estática; sigue un modelo de Autorización por Relaciones (ReBAC):

- Roles Contextuales: Un usuario no posee un rol único global. Sus privilegios dependen de su relación con un objeto, almacenada en tablas de vinculación.
- Asignación Dinámica: El servicio Core valida que el `entity_id` solicitado en una petición coincida con una asignación legítima del usuario dentro del JWT.
- Gestión de Concurrencia: Para escenarios de edición simultánea, el sistema implementa bloqueo optimista mediante versionado, asegurando que los cambios se apliquen solo si el usuario posee la versión más reciente del dato.

7.4 Sesiones y Revocación

El ciclo de vida de la sesión se gestiona mediante JTI (JWT ID) basados en **UUIDv7**.

7.4.1 Revocación Granular:

El JTI permite al Auth Service mantener un seguimiento preciso de las sesiones en la base de datos (RefreshSession), facilitando la revocación inmediata por usuario o dispositivo.

7.4.2 Transporte Seguro:

- Clientes Web: Implementación de cookies con directivas `HttpOnly`, `Secure` y `SameSite=Strict` para mitigar ataques XSS y CSRF.
- Clientes Mobile: Transporte vía cabeceras personalizadas (`X-Refresh-Token`), con validación estricta para evitar la suplantación de plataforma.

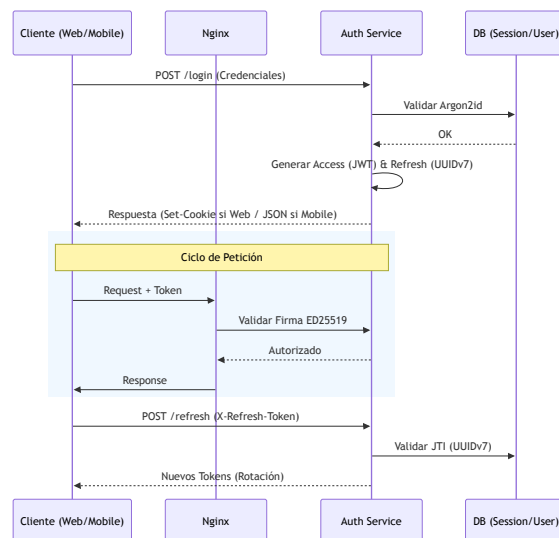


Figura 5: Flujo de autenticación, validación y rotación de tokens.

7.5 Seguridad de Contraseñas

El sistema utiliza el algoritmo **Argon2id**, estándar actual de OWASP. Este enfoque garantiza máxima resistencia contra ataques de fuerza bruta al combinar un salting único por usuario con parámetros de configuración de memoria (memory-hard), lo cual neutraliza la efectividad de intentos de descifrado mediante hardware especializado como GPUs.

8 Diseño de API

8.1 Convenciones REST

8.2 Estructura de la API

8.3 Gestión de errores

8.4 Paginación y filtrado

8.5 Documentación OpenAPI

9 Guía de desarrollo

9.1 Estructura del repositorio

9.2 Gestión de espacios de trabajo

9.3 Gestión de dependencias

9.4 Desarrollo local

9.5 Pruebas

9.6 Estilo de código

9.7 Contribución

10 Operaciones

10.1 Despliegue

10.2 Registro (Logging)

10.3 Monitoreo

10.4 Rendimiento

10.5 Mantenimiento

11 Evolución futura

11.1 Trabajadores en segundo plano (Background Workers)

11.2 Integración de datos abiertos

11.3 Búsqueda semántica

11.4 Integración de bases de datos vectoriales

11.5 Generación aumentada por recuperación (RAG)

11.6 Integración MCP

Bibliografía

- [1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004.